

UNIVERSITEIT ANTWERPEN
Academiejaar 2025-2026

Faculteit Toegepaste Ingenieurswetenschappen

4-Computerarchitectuur en besturingssystemen 2

Operating systems

Cursusdienst
UNIVERSITAS
Prinsesstraat 16
2000 Antwerpen
T +32 3 233 23 72
F +32 3 233 65 81
E info@cursusdienst.be
W www.universitas.be

**Bachelor of Science in de
industriële wetenschappen: elektronica-ICT**

STUDIEGIDS NR 1533FTICPA

Inhoudstabel

1. Voorwoord & inleiding.....	3
2. Planning	4
Oefening 1: tijdskritische en blocking functies combineren zonder OS (bare metal)	5
Oefening 2: Opzetten van een FreeRTOS project onder Atmel studio	6
Oefening 3: Multitasking	8
Oefening 4: Dynamische geheugenallocatie	10
Oefening 5: Stack variabelen, stack overflow detectie	12
Oefening 6: Communicatie tussen tasks: mutexes, atomic operations en queues	14
Oefening 7: Communicatie tussen tasks: een FIFO queue	16
Oefening 8: Communicatie tussen ISR's en tasks	17
Oefening 9: Meting van FreeRTOS task jitter	20
Oefening 10: motor hoekpositie- en snelheidsontroller	22
Project module 1: lokale user interface	25
Project module 2: oriëntatie meting d.m.v. een gyroscoop	28
Project module 3: lijnvolger	29
Appendix A FAQ	31
Bibliografie.....	32

1. Voorwoord & inleiding

In deze cursusmodule van het opleidingsonderdeel ‘4-Computerarchitectuur en besturingssystemen’ zullen we voortbouwen op de voorgaande modules waarbij we processor core en periferie van de Microchip XMEGA microcontrollers hebben leren gebruiken. Om complexere code te structureren komt een OS van pas. We zullen FreeRTOS, dit is een compact open source realtime OS, toepassen op deze processor familie. Dit zal ons toelaten meerdere parallelle taken (tasks) uit te voeren en te synchroniseren. Het realtime aspect laat toe meerdere taken uit te voeren met minimale responsietijd tussen bv een externe interrupt of een timer en de activatie van een task.

FreeRTOS maakt gebruik van een ‘fixed priority pre-emptive scheduling with time slicing algoritme’. Dit wil zeggen:

- **Fixed priority:** elke task krijgt een prioriteitsniveau dat niet veranderd wordt door de scheduler zelf. De gebruiker kan natuurlijk wel het niveau aanpassen.
- **Pre-emptive:** indien een andere task met hogere prioriteit gedeblokeerd wordt (in ‘ready’ toestand komt), wordt hier onmiddellijk op overgeschakeld, zonder een expliciete ‘yield’.
- **Time slicing:** na elke FreeRTOS tick interrupt wordt automatisch overgeschakeld naar een volgende task met dezelfde prioriteit volgens het round-robin principe.

Verder wordt aangehaald hoe op een veilige manier data kan uitgewisseld worden tussen deze tasks onderling, d.m.v. mutexen en critical sections. Ook wordt bestudeerd hoe verschillende types data (globale data, stack, dynamisch geheugen) beheerd worden onder het OS.

2. Planning

Hieronder vind je de planning voor de vijf labozittingen terug. Indien je merkt dat je begint achter te lopen op de planning, neem dan zelf het initiatief om zittingen bij te werken buiten de geplande contacturen.

Zitting 1:

Oefening 1: Tijdskritische en blocking functies combineren zonder OS

Oefening 2: Opzetten van een FreeRTOS project onder Atmel studio

Zitting 2:

Oefening 3: Multitasking

Oefening 4: Dynamische geheugenallocatie

Zitting 3:

Oefening 5: Stack variabelen, stack overflow detectie

Oefening 6: Communicatie tussen tasks: mutexes, atomic operations en queues

Zitting 4:

Oefening 7: Communicatie tussen tasks: een FIFO queue

Oefening 8: Communicatie tussen ISR's en tasks

Zitting 5:

Oefening 10: motor hoekpositie- en snelheidsontroller

Noot: Oefening 9 wordt NIET uitgevoerd.

Oefening 1: Tijdskritische en blocking functies combineren zonder OS (bare metal)

Documentatie:

- 4-computerarchitectuur en operating systems: periferie
- 4-computerarchitectuur en operating systems: core

Beschrijving:

We zullen code schrijven waarbij gelijktijdig een tijdskritische taak en een functie die wacht op een externe gebeurtenis via een polling loop zoals *scanf()* uitgevoerd wordt. In eerste instantie proberen we dit zonder OS.

Opdracht:



- Maak een kopie van je projectfolder gemaakt tijdens de periferie module van de cursus en noem deze LinebotOS-oef1.
- Open het nieuwe project, sluit vervolgens alle source files die nog open staan (window->close all). Dit om te vermijden dat er nog vensters openstaan met daarin bestanden uit het oude project! Klik op build->clean solution. **Doe dit voortaan in de volgende oefeningen ook altijd als je een kopie maakt van een bestaand project.**
- Schrijf code om een looplicht te maken dat de LED's in een looplicht patroon laat oplichten. Precies elke 500ms moet het looplicht een stap verder gaan.
- Gelijktijdig moet je programma op commando's reageren die via realterm ingegeven worden. Volgende tekst commando's moeten ondersteund worden: 'looplicht_links' en 'looplicht_rechts'. Deze twee commando's moeten respectievelijk het looplicht naar links en naar rechts laten lopen. Gebruik de *scanf()* functie om deze input op te halen.
- Belangrijk: het looplicht MOET voortdurend blijven lopen en mag niet stoppen terwijl op gebruikers commando's gewacht wordt. Hint: je kan een interrupt aan een Timer Counter koppelen.
- Evaluer, indien je de opdracht hebt voltooid, of deze manier van werken schaalbaar is naar complexere opdrachten waarbij meerdere functies gelijktijdig wachten op een gebeurtenis via een polling loop (zoals *scanf*). Noteer je conclusie in je portfolio onder 1a.

Oefening 2: Opzetten van een FreeRTOS project onder Atmel studio

Documentatie:

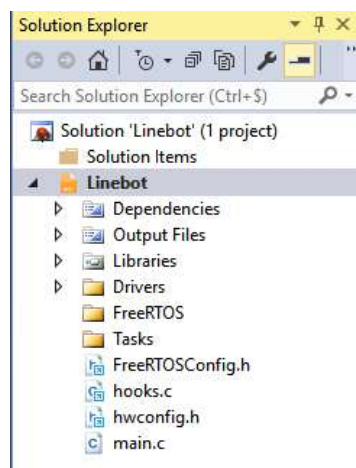
- PPT: FreeRTOS inleiding
- Mastering the FreeRTOS real time kernel; a hands on tutorial guide: hoofdstuk 1,2

Beschrijving:

In deze opdracht zal je de code die je hebt geschreven in de periferie module van de cursus integreren met FreeRTOS om als basis te dienen voor de volgende oefeningen.

Opdracht:

- Ga naar het Tools→Options→Projects→Miscellaneous menu. Zet hierin de optie 'Copy file to project when adding existing file' op 'False'. Dit zorgt ervoor dat als je een bestaand bestand toevoegt aan een project, hier geen extra kopie van gemaakt wordt.
- Maak een kopie van je projectfolder gemaakt tijdens de periferie module van de cursus en noem deze LinebotOS-oef2
- Vervang hierin de main.c file door de versie in LinebotOS-oef2.zip (via blackboard te downloaden). Deze main.c vormt een skelet *main()* functie. Hierin worden de hardware drivers en FreeRTOS geïnitieerd. Verder wordt er echter nog geen nuttige applicatie specifieke code uitgevoerd.
- Voeg in de 'Solution explorer' volgende bestanden toe aan de root van het project door op de projectnaam rechts te klikken, dan Add->existing item:



FreeRTOSConfig.h	FreeRTOS configuratie parameters. Deze parameters stellen o.a. tick rate en het al dan niet mee compileren van bepaalde features in. Door features weg te laten kan de grootte van de applicatie beperkt worden en performantie winst geboekt worden.
hooks.c	In dit bestand staan een aantal 'hook' functies. Deze functies worden door de gebruiker geprogrammeerd en bieden voornamelijk foutafhandelings functionaliteit. Voorbeelden hiervan zijn detectie van stack overflows en geheugen allocatie fouten. De functies in hooks.c worden door FreeRTOS rechtstreeks aangeroepen. Hier wordt later nog op teruggekomen.
memmap.c	memmap.c bevat een hulpfunctie om een geheugenkaart te tekenen. Wordt later in de oefeningen gebruikt.
FreeRTOS\heap_2.c	FreeRTOS dynamische geheugen allocatie code
FreeRTOS\list.c	FreeRTOS linked list implementatie. Wordt intern door FreeRTOS gebruikt.
FreeRTOS\port.c	FreeRTOS XMEGA processor specifieke code
FreeRTOS\queue.c	FreeRTOS queue code
FreeRTOS\tasks.c	FreeRTOS tasks code
FreeRTOS\timers.c	FreeRTOS timer code

- Doe een 'build' van het project (Build-->Build Solution) en controleer of er geen fouten zijn. Indien niet hebben we nu een FreeRTOS project dat klaar is om nuttige functionaliteit aan toe te voegen.

Oefening 3: Multitasking

Documentatie:

- PPT: Tasks
- Mastering the FreeRTOS real time kernel; a hands on tutorial guide: hoofdstuk 3

Beschrijving:

In deze opdracht zal je leren werken met FreeRTOS tasks om op deze manier in parallel meerdere taken uit te voeren.

Opdracht:

We werken verder op de code van oefening 2. Maak een kopie van de LinebotOS-oef2 directory en noem deze LinebotOS-oef3.

De bedoeling is nu de opdracht uit oefening 1 te hernemen, deze keer geïmplementeerd met FreeRTOS tasks.

- Schrijf een module (.C en .H file) om het looplicht te implementeren. Je kan je baseren op de bestanden 'TemplateTask.h' en 'TemplateTask.c' in de 'Tasks' subdirectory van je project. Elke module bestaat op zijn minst uit een initialisatie functie en een worker functie waarin de code staat die door de task wordt uitgevoerd. Gebruik de *xTaskCreate()* functie om een task aan te maken (zie de FreeRTOS reference manual & template). De prioriteit van de 'idle task' is `tskIDLE_PRIORITY`. Neem een prioriteitsniveau hoger dan dit basisniveau. Neem als stack grootte 'configMINIMAL_STACK_SIZE'. Deze twee laatste parameters zijn gedefinieerd in `FreeRTOSConfig.h`. Gebruik voorlopig de *_delay_ms()* functie om het looplicht te timen. Opgelet om de *_delay_ms()* functie correct te laten werken moet je de `hwconfig.h` header file includen bovenaan je C file. Dit is nodig gezien hierin de macro `F_CPU` (de CPU klokfrequentie) gedefinieerd is. De *_delay_ms()* functie in `util/delay.h` moet deze macro kunnen zien. Ook zoals eerder gemeld moet de optimization level van de compiler op 'O2' staan om de *_delay_ms()* functie correct te laten werken. (Project->LinebotTemplate properties->Toolchain->AVR/GNU C compiler->Optimization->Optimization level)



- Opgepast: worker functies mogen nooit beëindigd worden door een return of door tot het einde van de functie door te lopen. Ze moeten altijd een eindeloze lus bevatten OF beëindigd worden door een *vTaskDelete(0)* oproep. *vTaskDelete(0)* verwijdert de actuele task. Indien hier niet aan voldaan wordt, zal je code crashen.
- Vergeet niet de task te initialiseren en op te starten vanuit `main.c`. Roep de task init functie op, vlak onder de '//Init tasks' regel, voor de scheduler opgestart wordt. Als je dit niet doet, wordt de task code niet uitgevoerd.

- Schrijf nu een tweede task om via *scanf()* commando's via de terminal te accepteren om de richting van het looplicht in te stellen. Zorg dat deze task dezelfde prioriteit heeft als de looplicht task.
- Merk op: vanaf het moment dat je de terminal task activeert, is de timing van de *_delay_ms()* functie niet meer correct. Probeer de oorzaak hiervan te achterhalen. Noteer in je portfolio onder 3a waarom de timing van *_delay_ms()* niet meer klopt. Gebruik hiervoor volgende tools / gegevens:
 - Gebruik de *vTaskGetRunTimeStats()* functie. Deze functie geeft een overzicht van alle tasks en de toegekende processortijd. Let op. Om deze functie te gebruiken heb je een geheugenbuffer (512 bytes is voldoende) nodig om de data te stockeren. Gebruik uiteraard een lokale variabele voor deze buffer. Vergroot de stack van de task waarin je deze functie gebruikt met 512 bytes om plaats te maken voor deze variabele. De stack grootte kan aangepast worden via één van de parameters van de *xTaskCreate()* functie. Gebruik de *vTaskGetRunTimeStats()* functie vanuit de terminal task.
 - De *_delay_ms()* functie maakt intern gebruik van een 'delay loop'. Dit wil zeggen dat de processor een aantal keer een lus zal doorlopen waarvan het aantal processorcycli om deze te doorlopen gekend is.
 - Hou ook rekening met de interne werking van de *scanf()* functie in de terminal task. Deze input functie roept intern de functie *stdio_getchar()* aan in de USART driver (DriverUSART.c) om teken per teken op te halen.
- Verhoog nu de prioriteit van de looplicht task met '1' en voer de code terug uit. Wat gebeurt er? Verklaar. Zet deze verklaring mee in je portfolio onder 3b.
- Corrigeer de timing van het looplicht door middel van de *vTaskDelayUntil()* functie i.p.v. *_delay_ms()*. Noteer het verschil met *_delay_ms()* onder 3c. Noteer ook het verschil tussen *vTaskDelayUntil()* en *vTaskDelay()* onder 3d.
- Als je nu naar de processor belasting kijkt, zie je dat bijna alle processortijd naar de terminal task gaat. M.a.w. de terminal task vraagt ca 100% CPU belasting terwijl deze taak eigenlijk alleen wacht op input van de gebruiker. Dit probleem zullen we tijdens de volgende oefeningen aanpakken.
- In FreeRTOSConfig.h staan een aantal parameters die de werking van het OS bepalen. Pas de 'configUSE_PREEMPTION' parameter aan en zet deze op '0'. Hierdoor gaan we over van preemptive scheduling naar cooperative scheduling. Wat verandert er? Zet je bevindingen onder 3e.
- Zet na dit laatste experiment 'configUSE_PREEMPTION' terug op '1'.

Oefening 4: Dynamische geheugenallocatie

Documentatie:

- PPT: FreeRTOS inleiding
- Mastering the FreeRTOS real time kernel; a hands on tutorial guide: hoofdstuk 2

Beschrijving:

In deze oefening bekijken we hoe dynamische geheugenallocatie geïmplementeerd is op ons compact embedded platform, waar we rekening mee moeten houden t.o.v. complexere systemen met meer geheugen en performantie.

FreeRTOS ondersteunt dynamische geheugenallocatie op de 'heap'. De grootte van de heap wordt bepaald in FreeRTOSConfig.h met de configTOTAL_HEAP_SIZE parameter. Hou rekening met het feit dat bij de aanmaak van een task ook geheugen nodig is voor controlestructuren (TCB) en stack. Deze geheugenblokken worden op de FreeRTOS heap geplaatst. Om geheugen te alloceren / te dealloceren mogen we de klassieke *malloc()* en *free()* functie niet gebruiken. In de plaats komen *pvPortMalloc()* en *vPortFree()*. De syntax is identiek aan de oorspronkelijke functies. De functie *xPortGetFreeHeapSize()* geeft de hoeveelheid vrij heap geheugen weer. Indien er geprobeerd wordt geheugen te alloceren, dit niet mogelijk is omwille van een tekort aan (aaneengesloten) geheugen, wordt de *vApplicationMallocFailedHook()* functie automatisch aangeroepen in 'hooks.c'. De lopende tasks worden stilgelegd, de foutLED (LED 4) knippert snel (100ms aan, 100ms uit). Bovendien wordt een foutmelding over de terminal uitgestuurd.

Opdracht deel 1:

We werken verder op de code van oefening 3. Maak een kopie van de LinebotOS-oef3 directory en noem deze LinebotOS-oef4.

- Test *pvPortMalloc()* en *xPortGetFreeHeapSize()* door in de terminal task, na elke *scanf()* oproep een blok geheugen te alloceren. Print de resterende heap ruimte en het door *pvPortMalloc()* toegekende basisadres. Noteer de output van deze printopdracht in je portfolio onder 4a. Noteer ook:
 - In welke richting (stijgende / dalende adressen) worden de adressen op de heap toegekend?
 - Hoeveel bytes overhead is er bij het alloceren van een geheugenblok?
 - Wat gebeurt er indien meer geheugen opgevraagd wordt dan vrij is?
- Gebruik de 'MemMap()' functie om een geheugenkaart op het terminal venster weer te geven. Deze functie is gedefinieerd in memmap.c en is speciaal voor deze cursus geschreven, maakt geen deel uit van het standaard OS. Vergeet niet eerst memmap.h te includen. Controleer aan de hand van het toegekende adres via *pvPortMalloc()* dat het geheugenblok effectief op de heap gealloceerd is. Noteer de output van de memmap functie in je portfolio onder 4b. Leid uit de output van de memmap functie de grootte van de TCB af, noteer deze ook.

Opdracht deel 2:

Download en open het LinebotOS-oef4A project, te vinden op Blackboard. In dit project wordt onder andere een task genaamd 'memtask' aangemaakt. Deze task zal enkele variabelen aanmaken, hiervan het adres afprinten en uiteindelijk de memmap() functie aanroepen om daarna definitief in suspended state te gaan. Ook in main.c worden enkele variabelen aangemaakt.

Achterhaal de adressen waarop 'Var1, Var2, Var3, Var4, Var5, Var6' gestockeerd zijn. De adressen van Var4 tot Var6 worden op het terminal venster afgeprint in WorkerMemTask(). Var1 tot Var3 zijn echter niet toegankelijk in deze functie. Ze afprinten in main() is geen optie gezien op dat moment de USART nog niet geïnitieerd is. Je kan bijvoorbeeld gebruik maken van een watch window in breakpoint debugging mode om de adressen te bekomen. Merk ook op dat Var1 tot Var3 'volatile' gedeclareerd zijn. Dit heeft enkel en alleen als doel te vermijden dat deze variabelen door de compiler weggeoptimaliseerd worden, gezien ze niet echt gebruikt worden in de code.

Noteer onder 4C voor elke variabele in welke geheugensectie ze terechtkomt aan de hand van de geheugenmap bekomen met memmap(), verklaar ook waarom.

Opdracht deel 3:

Download en open het LinebotOS-oef4B project, te vinden op Blackboard. In dit project wordt een task, 'memtask' aangemaakt, welke elke 100ms de functie 'MemFunction' aanroept. Deze functie declareert een array, vult deze en geeft dan uiteindelijk een melding weer op de terminal. Je zal merken dat deze code correct werkt.

Open nu het LinebotOS-oef4C project, voer dit uit. Merk nu op dat na een tiental seconden de code beëindigd wordt met een memory allocation error. Noteer onder 4d wat het probleem is bij dit project. Hoe moet dit gecorrigeerd worden?

Opdracht deel 4:

Download en open het LinebotOS-oef4D project, te vinden op Blackboard. Bestudeer memtask.c. De heap wordt verdeeld in 6 stukken. Deze stukken worden meermaals gealloceerd en terug vrijgegeven zoals beschreven in de code. Voer de code uit.

- Merk op dat de uitvoering onderbroken wordt omwille van een memory allocation error tijdens stap 4. Open ook FreeRTOS\heap_2.c en bestudeer de beperkingen van de dynamische geheugenallocatie functies zoals hierin beschreven. Noteer nu onder 4e waarom de code faalt ongeacht het feit dat er eigenlijk voldoende vrij geheugen is.
- Verwijder heap_2.c uit het project en vervang deze door heap_4.c. Voer de code opnieuw uit. Als alles goed verloopt zal je nu voorbij stap 4 geraken. De code zal echter terug falen een eindje verder. Verklaar ook hier waarom dit gebeurt onder 4f in je portfolio.

Oefening 5: Stack variabelen, stack overflow detectie

Documentatie:

- PPT: Geheugenbeheer
- Mastering the FreeRTOS real time kernel; a hands on tutorial guide: hoofdstuk 2

Beschrijving:

Lokale variabelen, functieparameters / return waarden en functie terugkeeradressen worden op de stack gestockeerd. Elke task heeft een eigen stack, waarvan de grootte ingesteld wordt tijdens de aanmaak van de task via een parameter van de *xTaskCreate()* functie. Het is belangrijk dat deze stack voldoende groot is, gezien overtollige variabelen anders zullen geplaatst worden in een geheugengebied dat niet voorzien is als stackruimte en dusdanig in conflict kunnen komen met reeds ingenomen geheugenplaatsen.

Om dit op te vangen voorziet FreeRTOS twee controle mechanismen om stack overflows te detecteren. Het type controle mechanisme (1 of 2) kan in FreeRTOSConfig.h ingesteld worden d.m.v. de config CHECK_FOR_STACK_OVERFLOW parameter. Hou rekening met het feit dat deze controle extra tijd vergt tijdens de werking van de task scheduler. Ook is deze controle niet helemaal waterdicht. De controle wordt ook enkel uitgevoerd tijdens de oproep van de task scheduler (bv na een tick interrupt of nadat een task uit running state gaat, ...). Het werkingsprincipe van de controlemechanismen vind je op <https://www.freertos.org/Stacks-and-stack-overflow-checking.html>.

Opdracht:

We werken verder op de code van oefening 4. Maak een kopie van de LinebotOS-oef4 directory en noem deze LinebotOS-oef5.

- Test allocatie van variabelen op de stack uit door een recursieve functie (functie die zichzelf aanroept) aan te maken.
 - Deze keer gaan we de processor stilleggen elke keer de recursieve functie aangeroepen wordt, met behulp van een breakpoint en stap-voor-stap debugging.
 - In de recursieve functie print je de hoeveelheid vrije stack ruimte af en het adres (pointer) van één van de locale variabelen. De hoeveelheid vrije stack ruimte kan je bekomen d.m.v. de *vTaskGetTaskInfo()* en *xTaskGetCurrentTaskHandle()* functies. Zoek informatie op in de FreeRTOS reference manual. De 'task handle' is eigenlijk een pointer naar het begin van de TCB. Print ook deze task handle af.

- We gaan op het moment dat de breakpoint geactiveerd wordt, de geheugeninhoud van de task weergeven. Dit kan met behulp van een 'memory window'. Maak het memory window zichtbaar met behulp van Debug->Windows->Memory->Memory1. Geef als adres de 'task handle' pointer in. De geheugeninhoud op hogere adreslocaties bevat de TCB, de lagere adreslocaties bevatten de task stack. Merk op in welke richting (stijgende of dalende adressen) de stack aangroeit door de geheugeninhoud te observeren door meermaals de breakpoint te activeren (Debug->continue of F5). Noteer in je portfolio onder 5a, geef ook enkele screenshots van het memory window weer. Welke databyte waarde staat er op niet-gebruikte stack locaties? Hieronder wordt een voorbeeld van een memory window weergegeven, waarbij de task handle = 0x26C6

[illegible]

- Op het moment dat de grens van de beschikbare stack ruimte bereikt is, wordt de foutafhandelingsfunctie `vApplicationStackOverflowHook()` in `hooks.c` aangeroepen, worden alle taken stilgelegd, knippert LED 4 traag en wordt een foutmelding op de terminal getoond.

UITBREIDING:

- Tracht code te schrijven waardoor een stack overflow optreedt zonder dat dit door het stack overflow detectie mechanisme opgevangen wordt. Hiermee willen we de beperkingen van dit mechanisme aantonen.

Oefening 6: Communicatie tussen tasks: mutexes, atomic operations en queues

Documentatie:

- PPT: intertask communicatie
- Mastering the FreeRTOS real time kernel; a hands on tutorial guide: hoofdstuk 4,7

Beschrijving:

Communicatie van data tussen task worker functies onderling kan voor een aantal problemen op gebied van data integriteit zorgen. In deze oefening leggen we deze problemen bloot en onderzoeken we hoe dit kan opgelost worden.

Opdracht:

Maak gebruik van het code project voor oefening 6 dat je op blackboard terugvindt. Open dit project. In dit project vind je een bestand genaamd 'transfertask.c'. Hierin staan twee eenvoudige worker functies: *WorkerSendTask()* en *WorkerReceiveTask()*. De bedoeling is data van *WorkerSendTask()* over te dragen naar *WorkerReceiveTask()* met behulp van een globale variabele.



WorkerSendTask() schrijft alternerend de hexadecimale code 0x10101010 en 0x20202020 in een globale variabele, *Data*. *WorkerReceiveTask()* leest de globale variabele uit en controleert of deze werkelijk de waarde 0x10101010 of 0x20202020 bevat. Indien niet wordt een foutmelding afgeprint met de werkelijke waarde van de variabele *Data*.

- Voer de code uit. Merk op hoe de test onmiddellijk en stelselmatig faalt. *WorkerReceiveTask()* leest altijd de initialisatiewaarde '0' van 'Data' uit.
- Pas de declaratie van 'Data' aan door het voorvoegsel 'volatile' toe te voegen. Merk nu op dat als je de code uitvoert, 'Data' niet meer vastgepind staat op '0'. Sporadisch wordt de juiste waarde wel uitgelezen. Verklaar de werking van het voorvoegsel 'volatile', noteer in je portfolio onder 6a
- Zoals je merkt, is de communicatie nu ook nog niet perfect. Probeer te verklaren waarom foutieve waarden worden overgedragen. Schrijf ook deze verklaring bij in je portfolio, onder 6b.
- Los bovenstaand probleem op d.m.v. het gebruik van een mutex. Gebruik de *xSemaphoreCreateMutex()*, *xSemaphoreTake()* en *xSemaphoreGive()* functies.

Vergeet ook niet de header file 'semphr.h' te includen. Pas de mutex toe op elke individuele operatie op de gedeelde variabele (zoals `Data=0x20202020;`)

- Als je een mutex probeert te gebruiken die nog niet aangemaakt is met `xSemaphoreCreateMutex()` krijg je een foutmelding op je terminal te zien, beginnend met "*Assert in file ...*". Volg de verwijzing in de FreeRTOS code en zoek op wat er op dit regelnr. staat. Wat leid je hieruit af? Noteer onder 6c.
- Meet de tijd die nodig is in WorkerReceiveTask om de code te doorlopen om de semafoor op te halen, data in te lezen en semafoor terug te geven. Dit kan je doen m.b.v. de `portGET_RUN_TIME_COUNTER_VALUE()` functie. Deze functie geeft onder de vorm van een `uint32_t` teller de actuele processortijd weer. Elke processor cyclus wordt de teller met één verhoogd. Neem als meting altijd de kleinste waarde. Omwille van pre-emption kunnen er sprongen voorkomen in de metingen.
- Maak een kopie van de LinebotOS-oef6 folder en noem deze LinebotOS-oef6b. We gaan nu werken met atomaire operaties om data tussen twee tasks correct uit te wisselen.
- Gebruik de `taskENTER_CRITICAL()` en `taskEXIT_CRITICAL()` functies om de data variabele atomair weg te schrijven en in te lezen. Test of de code correct werkt.
- Meet ook hier de tijd die nodig is om de data variabele atomair in te lezen.
- Maak een kopie van de LinebotOS-oef6 folder en noem deze LinebotOS-oef6c. We gaan nogmaals de opdracht uitvoeren, deze keer met queues.
- Gebruik `xQueueCreate()`, `xQueueOverwrite()`, `xQueuePeek()` respectievelijk om een queue aan te maken, data op de queue te zetten en de laatst geschreven data uit te lezen. Maak de queue aan met plaats voor één data element.
- Meet de tijd om de data in te lezen van de queue.
- Vergelijk en noteer de drie tijdsmetingen en schrijf je conclusies in je portfolio onder 6d.

Oefening 7: Communicatie tussen tasks: een FIFO queue

Documentatie:

- PPT: intertask communicatie
- Mastering the FreeRTOS real time kernel; a hands on tutorial guide: hoofdstuk 4

Beschrijving:

Queues kunnen ingezet worden om in diverse werkingsmodi te functioneren (shared variable, FIFO, LIFO). In deze oefening werken we een toepassing met een queue in FIFO modus uit.

Opdracht:

We gaan de cursorstick driver uitbreiden met een task welke de cursorstick uitleest, telkens een actie gebeurt (cursor stick naar links, rechts, onder, boven, indrukken), wordt deze in een FIFO queue gestockeerd. Op deze manier kan een andere task die onregelmatig / aan een laag tijdsinterval uitgevoerd wordt toch de cursorstick uitlezen zonder dat er acties op de cursorstick verloren.

We werken verder op de code van oefening 3. Maak een kopie van de LinebotOS-oef3 directory en noem deze LinebotOS-oef7.

- Maak een nieuwe module aan op basis van TemplateTask.c.
- Maak een FIFO queue aan met een lengte van ca 5.
- Creëer een task waarin de cursorstick op vaste intervallen (bv 100ms) uitgelezen wordt. Telkens een beweging van de cursor stick gedetecteerd wordt, schrijf deze actie weg naar de FIFO queue. Zorg dat alleen veranderingen van de toestand van de cursor stick gedetecteerd worden. De cursor stick continu in één richting duwen zorgt dus slechts voor één schrijfoperatie naar de queue.
- Maak een publieke 'getter' functie in de module. Deze functie dient de eerst verstuurde actie van de queue te halen indien deze beschikbaar is.
- Breid de terminal task uit met een commando (bovenop de bestaande looplicht_links en looplicht_rechts) waarmee je de FIFO queue kan uitlezen via de getter functie. De inhoud van de FIFO queue dient op de terminal afgebeeld te worden.
- Test uit.

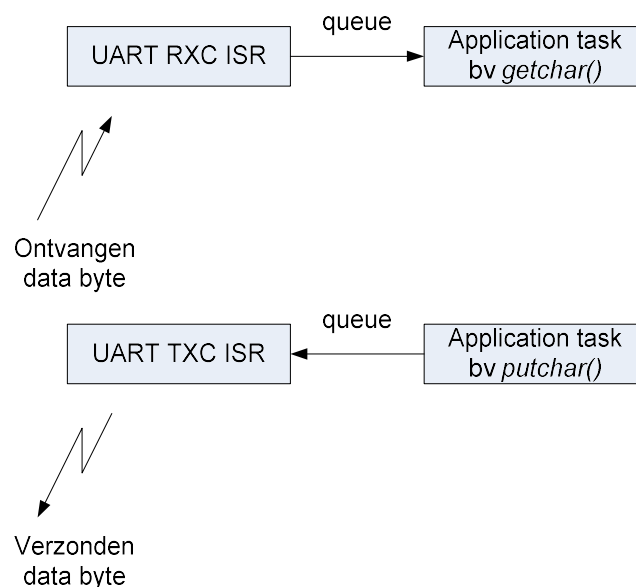
Oefening 8: Communicatie tussen ISR's en tasks

Documentatie:

- PPT: intertask communicatie
- Mastering the FreeRTOS real time kernel; a hands on tutorial guide: hoofdstuk 6,7

Beschrijving:

In de vorige oefeningen hebben we communicatie opgezet tussen twee tasks onderling. We zullen deze techniek nu uitbreiden om communicatie op te zetten tussen een ISR en een task. Dit is nodig als we willen communiceren tussen een hardware driver en een applicatie task. Een situatie waarbij dit van pas komt is bijvoorbeeld de UART driver. In oefening 3 hebben we gemerkt dat de huidige UART driver niet optimaal is. We kunnen de driver integreren in FreeRTOS door deze interrupt gestuurd te maken.

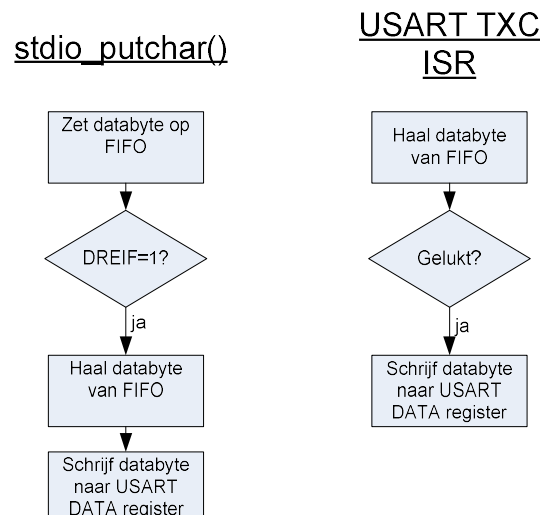


Net zoals bij intertask communicatie kunnen we semaforen, mutexen en queues gebruiken vanuit ISR's. Er moet echter op gelet worden dat de FreeRTOS functies met het achtervoegsel 'FromISR' gebruikt worden. Deze functies zijn speciaal geschreven om correct te werken vanuit een ISR. Om data op een queue te zetten is dit bijvoorbeeld *xQueueSendFromISR()*.

Opdracht:

We werken verder op de code van oefening 3. Maak een kopie van de LinebotOS-oef3 directory en noem deze LinebotOS-oef8.

- Begin met de UART receive functie uit te voeren met een queue. Hiervoor zal je het DriverUSART.c bestand moeten aanpassen. Gebruik de queue als FIFO. Dit heeft als voordeel dat inkomende data gebufferd wordt, niet verloren gaat zolang de FIFO lengte niet overschreden wordt. Begin met de UART driver te voorzien van een RXC interrupt + ISR en correct te configureren. Vervolgens kan je in de RXC ISR de inkomende data naar een queue wegschrijven. De data kan terug uit de queue gehaald worden in de `stdio_getchar()` functie. **Belangrijk:** de interrupts moeten als prioriteitsniveau 'low' hebben. Dit om te beletten dat de scheduler interrupt onderbroken wordt door een andere interrupt. Momenteel is het XMEGA FreeRTOS hier niet op voorzien. De nodige ISR vector identifiers staan in hwconfig.h gedefinieerd (USART_RXC_vect en USART_TXC_vect).
- Controleer de correcte werking van de UART driver. Ga nu ook terug de processorbelasting na. Wat merk je t.o.v. de vorige versie? Noteer in je portfolio onder 8a.
- Implementeer nu ook de transmit routine met een queue. Dit wordt iets complexer dan de ontvangst functie. Onderstaande flowchart vat het proces samen.
 - Als de UART momenteel geen data aan het verzenden is, moet data rechtstreeks verzonden worden vanuit de `stdio_putchar()` functie door naar het USART.DATA register te schrijven. Dit met data afkomstig van de FIFO queue. Of de UART momenteel al dan niet data aan het verzenden is, kan gecontroleerd worden aan de hand van de DREIF bit in het USART.STATUS register.
 - De volgende bytes dienen vervolgens verzonden te worden vanuit de TXC ISR door telkens één byte (indien voorhanden) van de queue te halen en in het USART.DATA register weg te schrijven.



- Schrijf de TXC ISR met en zonder *portYIELD_FROM_ISR()* functieaanroep (zie bv *xQueueSendFromISR()* in de FreeRTOS reference manual). Schrijf nu een lange string (bv 20 tekens) elke cyclus van de looplicht task. Meet met een smartscope op testpunt T6 (USARTE0.TXD lijn). Doe dit voor deze test met een queue lengte van '1'. Voeg de metingen toe bij je portfolio onder 8b, noteer tevens de verklaring.

Opgepast: om deze deelopdracht correct uit te voeren moet aandacht besteed worden aan enkele belangrijke randvoorwaarden:

- De USART driver moet geconfigureerd zijn voor een baudrate van 115200 bps.
- De task die het looplicht uitvoert moet een hogere prioriteit hebben dan de terminal task.
- Zoals hierboven vermeld: De UART transmit queue moet een lengte van '1' hebben. Dit is geen realistische situatie, maar maakt de invloed van *portYIELD_FROM_ISR* beter zichtbaar.

Oefening 9: Meting van FreeRTOS task jitter

Beschrijving:

Eén van de kerntaken van een RTOS is mogelijk te maken dat taken op vaste tijdsbasis kunnen uitgevoerd worden. De variatie van deze tijdsbasis, jitter genoemd, moet zo klein mogelijk zijn. We hebben tijdens vorige oefeningen tasks opgezet die om de x ms uitgevoerd worden met behulp van de `vTaskDelayUntil()` functie. In deze oefening zullen we de scheduling jitter opmeten, onderzoeken wat de invloed van bepaalde stoorfactoren zoals atomaire code of interrupts hierop is.

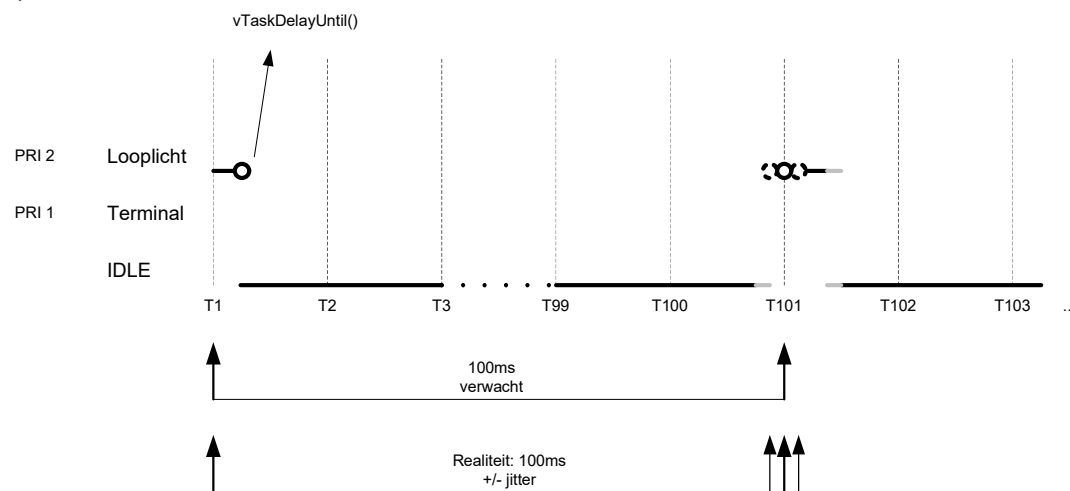
Opdracht:

We werken verder op de code van oefening 8. Maak een kopie van de LinebotOS-oef8 directory en noem deze LinebotOS-oef9.

- Herconfigureer de looplicht task om uit te voeren met een intervaltijd van 100ms. We krijgen onderstaande situatie, uitgaande van de veronderstelling dat de terminal task geen input van de gebruiker ontvangt en dus blocked/suspended is en wacht op input.

Task 1 (looplicht): Priority 2

```
while (1)
{
    ...
    vTaskDelayUntil(&lt;lw>,100);
}
```



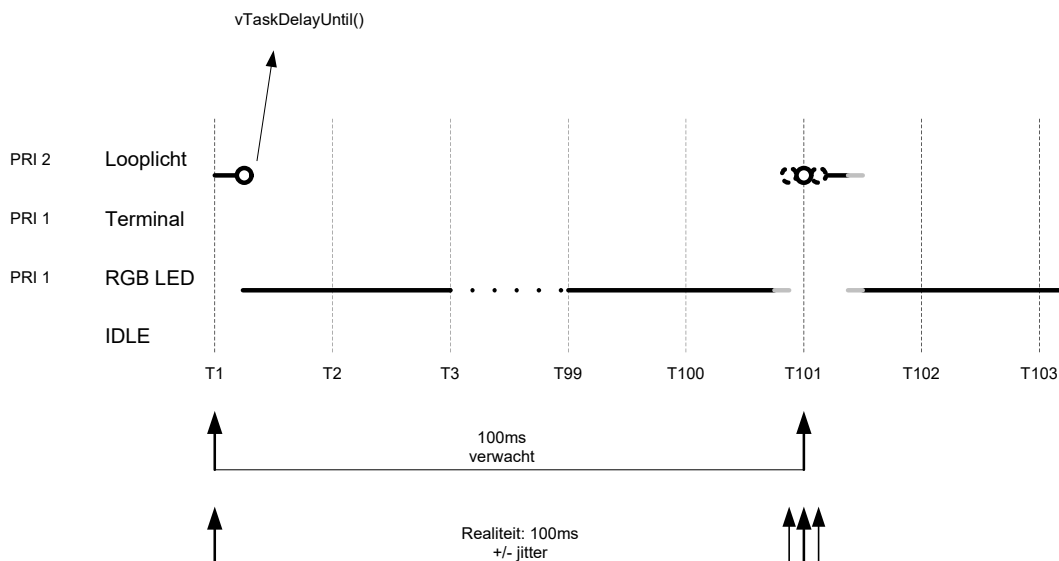
- Schrijf code in de looplicht task om de actuele en maximale tijdsjitter te meten. De jitter kunnen we uitdrukken als $t_{\text{jitter}} = (t_{\text{nu}} - t_{\text{vorige}}) - \Delta t_{\text{interval}}$. t_{nu} is het tijdstip aan het begin van de 100ms while lus. t_{vorige} is het tijdstip aan het begin van de 100ms while lus tijdens het vorige tijdslot. $\Delta t_{\text{interval}}$ is de verwachte intervalltijd (100ms dus). De actuele tijd kan met behulp van de `portGET_RUN_TIME_COUNTER_VALUE()` functie opgevraagd worden.
- Schrijf de jitter en maximale jitter weg naar het terminalvenster. Noteer deze tijden in je portfolio onder 9a.
- Maak vervolgens een extra RGBLED task met een prioriteitsniveau lager dan de looplicht task. Integreer hierin de PL9823 RGB LED driver die je hebt geschreven tijdens de module ‘werking van de core’ of de PL9823 driver die je onder de ‘Drivers’ map vindt van je project. Schrijf non-stop data naar de RGB LED’s zoals in onderstaande figuur weergegeven. Noteer ook weer jitter en maximale jitter, ditmaal onder 9b. Je zal ook merken dat de RGB LED’s niet feilloos aangestuurd worden. Af en toe treden onregelmatigheden op zoals LED’s die in helderheid / kleur fllikkeren. Noteer in je portfolio onder 9c wat hiervan de oorzaak is.
- Los bovenstaand probleem op door de RGB LED code atomair uit te voeren. Meet opnieuw jitter en maximale jitter en noteer deze onder 9d.
- Noteer je conclusie betreffende de drie jitter metingen onder 9e.

Task 1 (looplicht): Priority 2

```
while (1)
{
    ...
    vTaskDelayUntil(&lwt,100);
}
```

Task 2 (RGB LED): Priority 1

```
while (1)
{
    DriverPL9823Set(...);
}
```

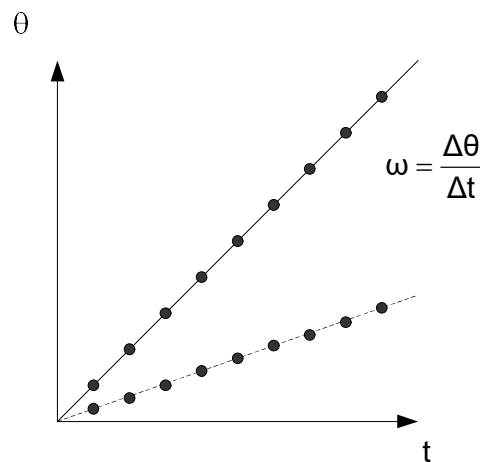


Oefening 10: motor hoekpositie- en snelheidsontroller

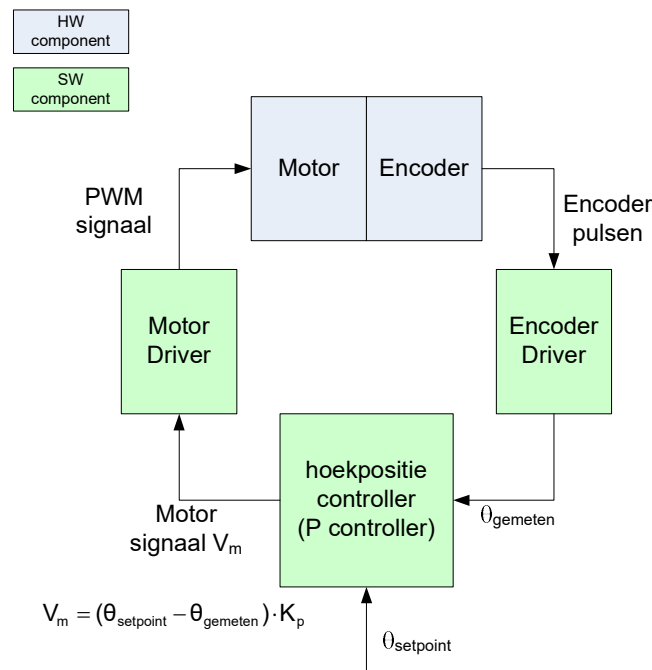
Beschrijving:

In deze oefening gaan we de twee motoren van de robot aansturen om beiden gecontroleerd aan een bepaalde gewenste snelheid te laten roteren. We hebben reeds de motoren in snelheid kunnen variëren d.m.v. PWM aansturing. De rotatiesnelheid en daardoor ook de voortbewegingssnelheid van de robot is echter onderhevig aan variaties in belasting en wrijving. Om nauwkeurig de snelheid te kunnen regelen van de twee motoren, wat o.a. noodzakelijk is om in een rechte lijn te kunnen rijden, hebben we een gesloten lus systeem nodig. Hierbij meten we continu de rotatiesnelheid aan de hand van de optische encoders op de motorassen en sturen we deze bij.

Om de snelheidsregelaar op te bouwen gaan we via een omweg werken: We kunnen de hoeksnelheid van elke motor definiëren als $\omega = \frac{d\theta}{dt}$ of tijdsdiscreet $\omega = \frac{\Delta\theta}{\Delta t}$. Elke tijdstap Δt gaan we de gewenste hoekpositie van de motor verhogen met een constante $\Delta\theta$. Hoe hoger deze laatste constante, hoe sneller de motor draait.



Om dit te verwezenlijken, moeten we de hoekpositie van de motor kunnen bepalen en controleren. Dit gaan we doen aan de hand van een eenvoudige proportionele controller (P controller) waarvan de werking verduidelijkt wordt aan de hand van onderstaand schema. Eventueel zou de werking nog kunnen uitgebreid worden tot een volwaardige PID controller om de nauwkeurigheid en reactiesnelheid te vergroten. Dit valt echter buiten het bestek van deze opdracht.



Opdracht:

- Pas de motor driver aan, beveilig de *DriverMotorSet()* functie met behulp van een mutex zodat deze nooit gelijktijdig vanuit meerdere tasks kan aangeroepen worden.
- Pas de encoder driver aan, herschrijf de *DriverMotorGetEncoder()* functie zodat de huidige encoder hoekpositie veilig kan uitgelezen worden door o.a. de hoekpositie controller task. Zorg ook dat de hoek in graden wordt teruggegeven i.p.v. uitgedrukt in encoder pulsen.
- Maak een task aan voor de hoekpositie P-controller. In deze task implementeer je de P-controllers voor beide motoren. Deze task communiceert met de motor- en encoderdriver. Een tijdstap van 10 ms is normaal voldoende. Test de werking van de P-controller door de motor bv elke 2 seconden een stap van 90° te laten maken. Indien correct geschreven en afgeregeld, zal de motor een nauwkeurige stap maken, zonder teveel naslingering. Naslingering treedt op bij een te hoge K_p regelconstante. Indien K_p te laag is, zal er teveel 'rek' op de hoekpositie zijn: de controller regelt niet nauwkeurig en snel genoeg. Normaal zal het systeem weerstand bieden als je handmatig probeert het wiel te verdraaien: de controller probeert altijd de gewenste hoekpositie te behouden.

- Om de gewenste K_p te vinden gaan we de Ziegler-Nichols methode hanteren.
 - Start met een lage K_p
 - Verhoog stelselmatig K_p totdat de regelaar uit zijn eigen begint te oscilleren.
 - Deze kritische K_p noemen we K_c .
 - Neem nu $K_p = K_c/2$. Dit is in principe de optimale K_p . In de praktijk zal je deze meestal echter nog manueel moeten bijtunen.
- Schrijf nu een task voor de hoeksnelheids aansturing. In deze task zal je elke tijdstap nieuwe hoekpositie setpoints doorgeven aan de hoekpositie controller task. Voorzie dus correcte intertask communicatie. Voorzie ook een functie om de hoeksnelheid van de twee motoren te kunnen instellen vanuit een andere task. Ook hier is correcte intertask communicatie cruciaal.
- Test de volledige werking uit door de robot in een rechte lijn rechtdoor te laten rijden.

Enkele algemene tips om de controller stabiel te krijgen:

- Omwille van de lage resolutie van de encoders is het moeilijk om een correcte en stabiele regeling te krijgen bij lage snelheden. Dit volgt uit het feit dat bij lage snelheid de tijdsperiode tussen de encoderpulsen lang is en de regelaar dus gedurende een lange tijdsperiode geen update van de hoekpositie krijgt. Test de werking dus uit bij voldoende hoge snelheid (streefwaarde $> 90^\circ/\text{s}$).
- De motor driver heeft een sterk niet-lineaire responsie o.a. door statische wrijving waardoor de motor pas zal beginnen draaien bij een PWM duty cycle $> 40\%$. Probeer de responsie van de driver te lineariseren. Dit zal de stabiliteit van de P(ID) controller sterk verbeteren.

UITBREIDING:

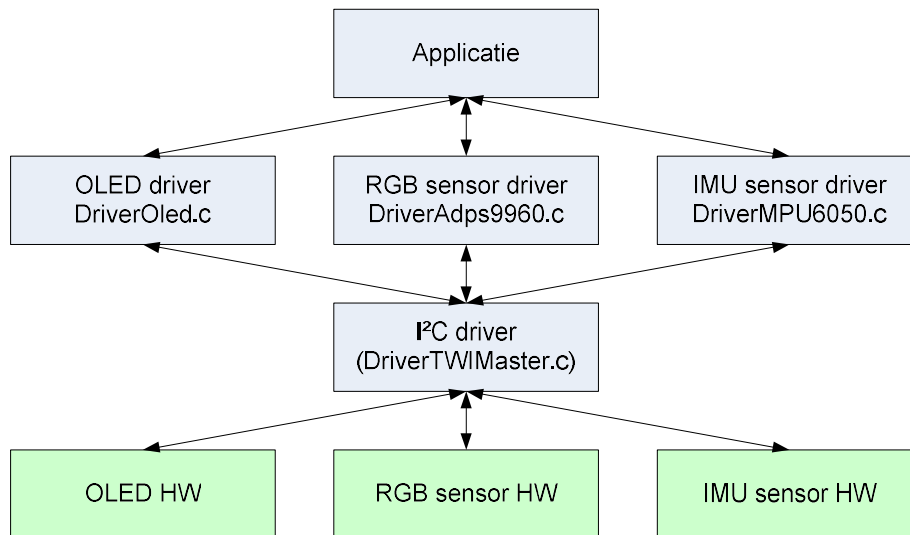
- Zorg dat je de robot over een bepaalde afstand kan laten rijden, daarna automatisch stoppen. Gebruik de encoder informatie hiervoor.
- Implementeer andere bewegingen zoals een rotatie van de robot over een bepaalde hoek.

Project module 1: lokale user interface

Beschrijving:

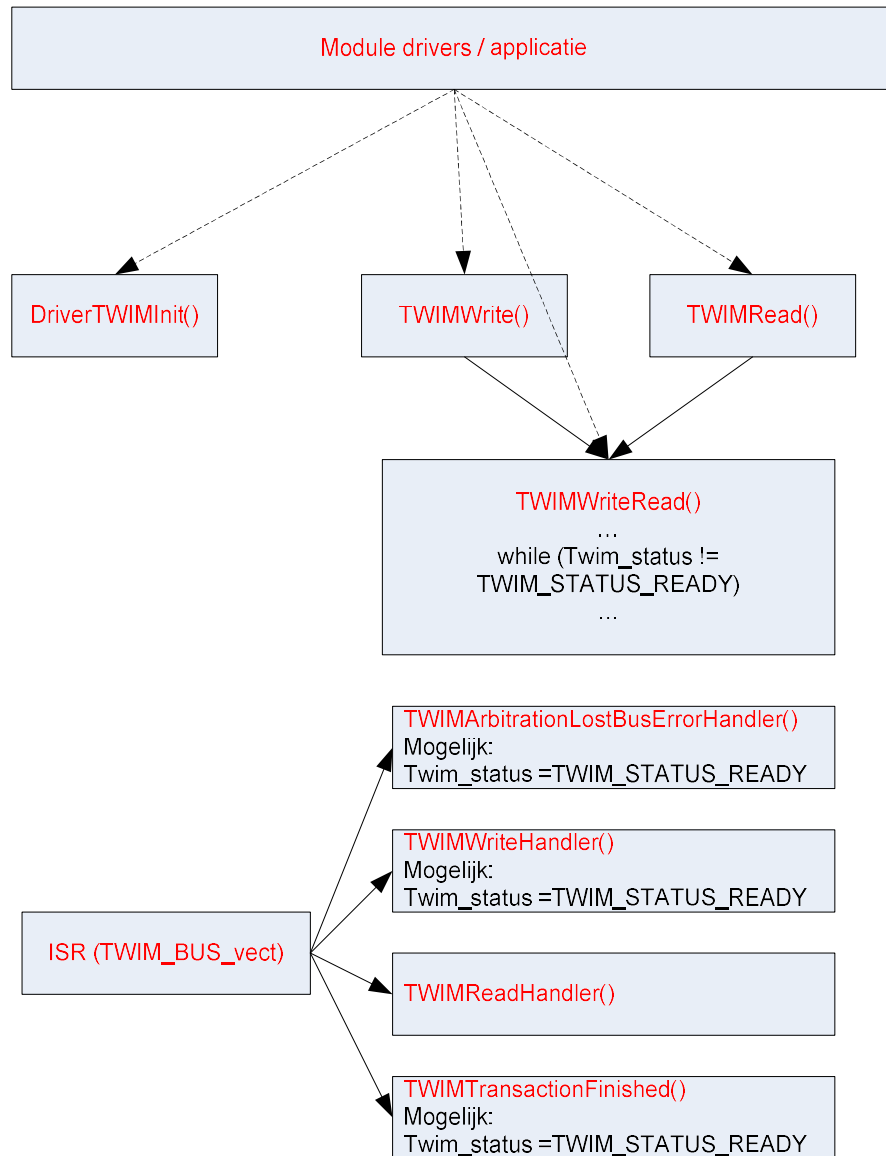
Op de robot is een klein monochroom grafisch OLED display aanwezig met een resolutie van 128x64 pixels. Dit display is via een I²C bus, of ook TWI (Two Wire Interface) genaamd in de processor manual, aangesloten op de microcontroller. I²C is een seriële bus met een maximale transfersnelheid van 400 kbps in combinatie met de modules gebruikt in onze toepassing. Deze bus is ontworpen om binnen een toestel verscheidene intelligente modules aan te spreken vanuit een centrale microprocessor. Elke module kan via een uniek adres geselecteerd worden. Vanuit de processor (master) kan data onder de vorm van een of meerdere bytes naar een reeks opeenvolgende registers (geheugenlocaties) op de geadresseerde module (slave) geschreven worden. Data kan eveneens uitgelezen worden op aanvraag van de processor.

Op de figuur hieronder wordt de I²C driver structuur weergegeven. De applicatie spreekt de hardware modules aan via module specifieke drivers. Deze spreken op hun beurt de I²C bus aan via de I²C driver (DriverTWIMaster.c). Gezien de lage datatransfersnelheid van de bus zal de I²C driver regelmatig moeten wachten op voltooiing van de datatransfers.



De I²C driver in het project is echter nog niet geïntegreerd in FreeRTOS en zal hierdoor net zoals de oorspronkelijke UART driver overtollige CPU belasting met zich meebrengen en niet gelijktijdig kunnen aangesproken worden vanuit meerdere tasks.

Gezien de complexiteit van de I²C driver, wordt deze hieronder schematisch samengevat:



De applicatie initialiseert de I²C driver via *DriverTWIMInit()*. De module drivers starten een I²C transfer op via *TWIMWrite()*, *TWIMRead()* of *TWIMWriteRead()*. Intern roepen *TWIMWrite()* en *TWIMRead()* de functie *TWIMWriteRead()* op. Het zal deze functie dan ook zijn die de eigenlijke datatransfer opstart. De functie zal vervolgens wachten totdat *Twim_status*=*TWIM_STATUS_READY* door te wachten in een polling loop. *Twim_status* wordt aangepast na voltooiing van de transfer in één van de subfuncties van de bijhorende ISR.

Om de driver te integreren in FreeRTOS zal je enerzijds moeten zorgen dat de driver niet gelijktijdig door meerdere tasks gebruikt wordt. Anderzijds zal je de polling loop in *TWIMWriteRead()* moeten vervangen om overbodige CPU belasting te vermijden.

Verder is, zoals reeds in het periferie onderdeel van het practicum gezien, een cursorstick gemonteerd. Ook deze cursorstick zal geïntegreerd moeten worden in FreeRTOS om denderij en conflictvrij de stick te kunnen uitlezen vanuit meerdere tasks.

Met de twee bovenstaande elementen kan een eenvoudige user interface opgebouwd worden waarmee de robot bediend kan worden.

Volgende elementen moeten minimaal aanwezig zijn in de UI:

- Hoofdmenu met selecteerbare items (subpagina's)
- Volgende items geïmplementeerd:
 - Weergave van de CPU status (CPU belasting, vrij geheugen, aantal tasks)
 - Activatie en monitoring van de functionaliteit van de andere project modules. Zolang de modules nog niet geïmplementeerd zijn, roep je blanco / gesimuleerde functies aan. Zorg dat je de interface met de andere modules afsprekt met je medestudenten.

Opdracht:

- Maak een OLED menu task aan. In deze task wordt de OLED UI afgehandeld. Initialiseer in deze task het display m.b.v. de *DriverOLEDInit()* functie. Zet het display in 'landscape inverted' mode. De parameters van de OLED display functies vind je terug in *DriverOled.h*. De meeste OLED display functies schrijven naar een interne RAM buffer op de microcontroller. Pas als de *DriverOLEDUpdate()* functie wordt aangeroepen, wordt de RAM buffer naar het display weggeschreven. Test enkele display functies uit.
- Integreer de I²C driver op een correcte manier in FreeRTOS zoals in de beschrijving weergegeven.
- Verbeter de cursorstick driver door ontdendering toe te voegen. Communiceer met de tasks die gebruik maken van de cursorstick door elke keer dat een actie wordt uitgevoerd (bv cursor stick naar links, rechts, boven, ...), deze actie via een queue door te geven.
- Bouw de UI op zoals in de beschrijving opgelijst.

Project module 2: oriëntatie meting d.m.v. een gyroscoop

Beschrijving:

De robot bevat een IMU-module. Deze IMU-module is opgebouwd rond een MPU-6050 chip waarin een 3-assige accelerometer en 3-assige gyroscoop geïntegreerd is. De gyroscoop meet de hoeksnelheid rond X,Y en Z as. Gezien de robot altijd vlak op tafel staat, is alleen de Z-as (vertikale as) informatie nuttig voor ons. We kunnen de hoekpositie θ op tijdstip t_m bepalen door integratie van de hoeksnelheid. Ideaal is $\theta(t_m) = \int_0^{t_m} \omega(t) \cdot dt$. In realiteit zit er op de gemeten hoeksnelheid een offset fout, waardoor de berekende hoekpositie snel zal afdrijven:

$$\theta(t_m) = \int_0^{t_m} (\omega(t) + \omega_{offset}) \cdot dt$$
$$\theta(t_m) = t_m \cdot \omega_{offset} + \int_0^{t_m} \omega(t) \cdot dt$$

Deze offset fout moet dus gecompenseerd worden. Dit doe je door ω_{offset} op te meten bij opstart van het systeem, af te trekken van de gemeten ω . Je kan in eerste instantie veronderstellen dat deze fout constant blijft.

De integratie van de hoeksnelheid kan je op de microcontroller uitvoeren d.m.v. numerieke integratietechnieken zoals de trapeziumregel.

De MPU-6050 is verbonden met de microcontroller via een gedeelde I²C bus, net zoals het OLED display in project module 1. Lees de betreffende tekst door. Om de hoeksnelheden uit te lezen is reeds een driver geschreven. Deze driver vind je terug in DriverMPU6050.c en DriverMPU6050.h. Deze maakt net zoals het OLED display gebruik van de TWI / I²C driver in DriverTWIMaster.c.

Opdracht:

- Maak een gyro task aan. Hierin zal je de hoekpositie d.m.v. numerieke integratie berekenen. Maak in eerste instantie gebruik van de oorspronkelijke DriverTWIMaster.c.
- Integreer de I²C driver op een correcte manier in FreeRTOS zoals in de beschrijving weergegeven.
- Schrijf een interface functie die door een andere task gebruikt kan worden om op een veilige manier de actuele hoekpositie uit te lezen, berekend door de gyro task. Test deze functie uit door in een bijkomende task continu de hoekpositie af te printen op het terminal venster. Controleer in deze task ook of de processor belasting correct is. Deze zou minder dan 10% moeten bedragen.

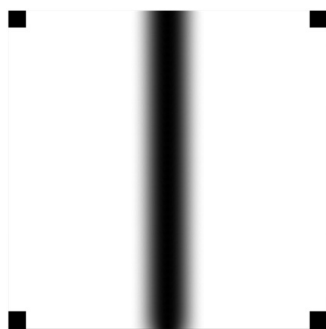
UITBREIDING:

- Je zal merken dat de berekende hoekpositie na verloop van tijd toch nog afdrijft. Dit wordt veroorzaakt door het feit dat ω_{offset} niet helemaal constant is in de tijd (o.a. door temperatuursafhankelijkheid). Bedenk en implementeer een manier om de drift van de hoekpositie te verbeteren.

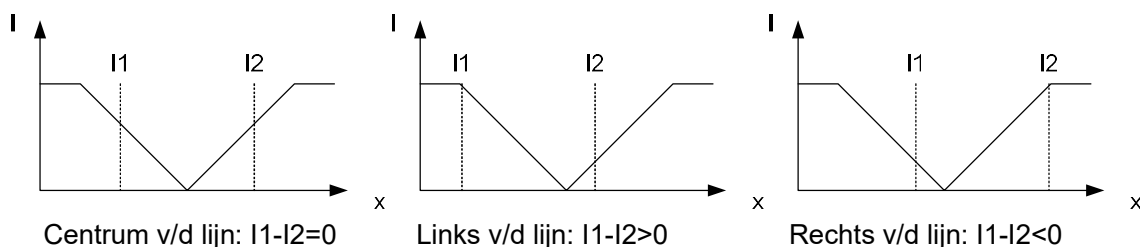
Project module 3: lijnvolger

Beschrijving:

Je zal in deze module code schrijven om de robot een lijn geprint op tegels te laten volgen. Deze lijn heeft geen harde contouren maar is als een gradiënt uitgevoerd:



Dit laat ons toe met behulp van de fotoreflectie sensoren aan de onderkant van de robot te meten hoe ver we van het centrum van de lijn verwijderd zijn. Neem drie situaties, waarbij we telkens de intensiteitswaarde I_1 meten van de linker sensor en de intensiteitswaarde I_2 van de rechter sensor:



We kunnen nu een eenvoudige proportionele regelaar maken die de snelheid van beide motoren differentieel aanstuurt om de robot naar het centrum van de lijn te sturen:

$$V_{Mlinks} = V_{cst} + (I_1 - I_2) \cdot K_p$$

$$V_{Mrechts} = V_{cst} - (I_1 - I_2) \cdot K_p$$

V_{Mlinks} en $V_{Mrechts}$ zijn de motorsignalen die we sturen naar de motoren d.m.v. PWM aansturing.

V_{cst} is het motorsignaal als de robot rechtdoor rijdt. Door dit signaal te variëren, kunnen we de gemiddelde snelheid van de robot ruwweg bepalen.

K_p is een regelconstante die bepaalt hoe agressief de robot bijstuurt. Deze constante moet afgeregeld worden zodat de robot snel genoeg reageert op afwijkingen. Indien de constante echter te hoog is, zal de regeling onstabiel worden en beginnen oscilleren of naslingeren.

Merk op dat we de motoren niet nauwkeurig in snelheid aansturen. Om dit toch te doen, zouden we op de snelheidsregelaar van module 2 moeten verderbouwen. Dit valt echter buiten het bestek van het project.

Opdracht:

- Pas de motor driver aan, beveilig de *DriverMotorSet()* functie met behulp van een mutex zodat deze nooit gelijktijdig vanuit meerdere tasks kan aangeroepen worden.
- Pas de ADC driver aan zodat deze binnen FreeRTOS past.
 - Zorg dat *DriverAdcGetCh()* nooit gelijktijdig vanuit meerdere tasks kan aangeroepen worden.
 - Ga over naar een interrupt gestuurde werking. Zorg voor adequate communicatie tussen de ISR en de task die *DriverAdcGetCh()* heeft opgeroepen.
- Maak een lijnvolger task aan. Implementeer hierin het algoritme zoals in de beschrijving hierboven aangegeven. Start met een lage Kp, verhoog deze stelselmatig totdat de robot correct de lijn volgt. Hou om te testen eventueel de robot handmatig boven de lijn, beweeg deze lateraal om het correcte gedrag van de lijnvolger uit te testen.
- Voorzie een functie om de lijnvolger te starten / stoppen vanuit een andere task (bv UI).

UITBREIDING:

- Zorg ervoor dat de robot stopt op het einde van een lijnsegment.

Appendix A FAQ

printf() print alleen de eerste regel van een tekst af.

printf() stopt met printen na een 'LF of linefeed' karakter --> gebruik puts() i.p.v. printf()

gets() reageert niet op een 'return / enter' in realterm / putty.

gets() keert slechts terug na het ontvangen van een 'LF' dit in tegenstelling met scanf() welke terugkeert na 'CR' / 'LF' of spatie.

Je kan een LF in realterm geven door op CTRL-J te drukken of CTRL-ENTER.

_delay_ms() geeft een verkeerde delay tijd (te lang / te kort)

De optimization level van de compiler moet op 'O2' staan om de _delay_ms() functie correct te laten werken. (Project->LinebotTemplate properties->Toolchain->AVR/GNU C compiler->Optimization->Optimization level).

Het lukt niet om via breakpoint debugging breakpoints te zetten / variabelen te bekijken.

De optimization level van de compiler moet op 'O0' staan om correct met breakpoints te werken. Hou rekening met het feit dat hierdoor de _delay_ms() functie niet meer correct werkt.

Hoe een globale variabele aanmaken toegankelijk vanuit meerdere C bestanden?

Declareer de variabele in één C bestand. (bv int counter)

- Declareer de variabele opnieuw in de header file horend bij het C bestand, met het extern keyword (extern int counter) .
- Elk C bestand dat toegang wil hebben tot de variabele moet deze header file toevoegen m.b.v. een #include " " regel.

Ik krijg een stack overflow melding. De naam van de betreffende task wordt echter niet afgeprint.

Vaak is de oorzaak een worker functie zonder eindeloze while lus of een return / break commando in deze worker functie. Een worker functie mag nooit beëindigd worden, tenzij de task eerst correct wordt afgesloten m.b.v. *vTaskDelete()*.

Bibliografie

1. FreeRTOS reference manual:
https://www.freertos.org/Documentation/FreeRTOS_Reference_Manual_V10.0.0.pdf
2. Mastering the FreeRTOS real time kernel; a hands on tutorial guide:
https://freertos.org/Documentation/161204_Mastering_the_FreeRTOS_Real_Time_Kernel-A_Hands-On_Tutorial_Guide.pdf
3. I²C bus specification and user manual: <https://www.nxp.com/docs/en/user-guide/UM10204.pdf>